به نام خدا

دانشکده‌ی مهندسی برق و کامپیوتر

یادگیری تعاملی - پاییز ۱۴۰۳ – ۱۴۰۴

تمرین سری دوم

فاطمه محمدی – ۸۱۰۱۹۹۴۸۹

بخش اول: ۲
سوال اول: ۲
سوال دوم: ۳
سوال سوم: ۴
بخش دوم: ۵
سوال چهارم: ۵
سوال پنجم: ۶
بخش سوم: ۸
سوال ششم: ۸
سوال هفتم: ۸
الف) در فایل نوت بوک ۸
ب) ۸
ج) در پوشه مربوطه قرار گرفته اند. ۸
سوال هشتم: ۸
بخش چهارم: ۹
سوال نهم: ۹
سوال دهم: ۹
منابع: ۱۰

بخش اول:

سوال اول:

در روشهای یادگیری مبتنی بر ارزش، تمرکز روی تخمین یک تابع $f(a,s,w)$ و پارامترهای w هستیم تا بتوانیم $v(s)$ (و یا $Q(s,a)$) را تخمین بزنیم و یا به اصطلاح parameterized کنیم. از طرف دیگر در روشهای مبتنی بر یادگیری سیاست به صورت مستقیم، در تلاش هستیم با جستجو در فضای سیاست، به صورت مستقیم و بدون یادگیری Q-valueها سیاست بهینه را پیدا کنیم و به عبارتی لازم است که سیاست را parameterized کرد.

در ادامه (جدول زیر) به صورت خلاصه به تفاوتها و ایده های اصلی هر دو روش میپردازیم:

ویژگی / روش	یادگیری مبتنی بر ارزش	یادگیری مبتنی بر سیاست
ایده اصلی	یادگیری و تخمین $\boldsymbol{v(s)}$ (و یا $\boldsymbol{Q(s,a)}$) برای بدست اوردن سیاست	با parameterized کردن سیاست و یادگیری این سایست پارامتری $\boldsymbol{\pi_\theta(a \mid s)}$ ، مشاهدات را به احتمال انتخاب اکشنها (یا توزیعی بروی اکشن ها) نگاشت کند
الگوریتم های رایج	• DQN (از شبکه‌های عصبی برای تقریب $Q(s,a)$ استفاده میشود.) • Double DQN • Dueling DQN	• REINFORCE: الگوریتم ساده گرادیان سیاست مونت کارلو. • PPO (Proximal Policy Optimization) • TRPO (Trust Region Policy Optimization)
برخی از مزایا و معایب	- برای مسائل با مجوعه اکشن پیوسته (یا گسسته با فضای اکشن بزرگ) مناسب نیست. + عموما میتواند sample- efficienty بهتر و بیشتری داشته باشد	+ قابلیت مدیریت فضاهای عمل پیوسته و با ابعاد بالا + یادگیری سیاست در محیط‌های تصادفی مفیدتر است + خوش دست بودن روشهای مبتنی بر سیاست برای یادگیری اجتماعی و افزودن قید به سیاست

اما اصلی ترین تفاوتی که میتوانیم نام ببریم و جز مزیتهای روش مبتنی بر سیاست محسوب میشود ، **خوش دست بودن روشهای مبتنی بر سیاست برای یادگیری اجتماعی و افزودن قید به سیاست** میباشد.

در واقع در بسیاری از سناریوها ما علاقه داریم تا یک سری از قیود را در سیاست در نظر بگیریم، برای مثال در مسائل human in the loop مانند Autonomous Driving، افراد ممکن است نخواهند به صورت کامل از سیاست در نظر گرفته شده در عامل بهره ببرند و بخواهند عامل کمی به صورت شخصی سازی شده تر رفتار کند و عامل باید رفتار خود را به سمت افزایش پاداش دریافتی در انسان متمایل کند ولی در عین حال باید از یک سری اکشنهای پر خطر مانند فشردن گاز زمانی که در نزدیک یک ماشین هست، دوری کند و این چنین محدودیت ها و قید ها که عموما در فضای پالیسی قابل بیان هستند را میتوان به راحتی در روشهای یادگیری مبتنی بر سیاست قرار داد.

از جمله دیگر مثالهایی که برای کاربرد و نیاز ما به روشهای یادگیری مبتنی بر سیاست میتوان نام برد، یادگیری احتماعی است که در ان عامل پس از شناسایی عاملهای مناسب و نامناسب در محیط باید سعی کند سیاست خود را شبیه سیاسیت عاملهایی که فکر میکند خوب است بکند و از سیاست عامل هایی که بنظر نامناسب هستند دوری کند و این را مجددا میتوان در فضای سیاست به راحتی تعریف کرد و به عنوان یک Secondary Objective به تابع هزینه اضافه کرد، در حالی که تعریف این قیود در روش های مبتنی بر ارزش بسیار دچارتر است.

یکی دیگر از دلایل ترجیح سیاست به ارزش، مربوط به محیطهایی با تعداد زیادی عمل یا فضاهای عمل پیوسته است.
برای اینکه بتوانیم در روش یادگیری مبتنی بر ارزش بهترین عمل را انتخاب کنیم، باید یک مسئله بهینه سازی حل کنیم تا a را پیدا کنیم که $Q(s,a)$را به حداکثر برساند .در مثال بازی های آتاری که تعداد کمی عمل گسسته دارند، این مسئله مشکلی نیست و ما مقادیر Q را برای تمام اعمال ممکن تقریب میزنیم و عملی که بزرگ‌ترین مقدار Q را داشته باشد انتخاب میکنیم. اما اگر فضای عمل ما پیوسته باشد (برای مثال، زاویه فرمان یک خودرو)، حل مسئله بهینه سازی سختتر میشود. زیرا $Q(s,a)$ معمولا توسط یک شبکه عصبی بسیار غیرخطی نمایش داده میشود و پیدا کردن a هر مقدار این تابع را بیشینه کند، پیچیده است.در چنین مواردی، کار کردن مستقیم با سیاست به جای ارزش بسیار عملیتر است.

- روش‌های Actor-Critic (ترکیب هر دو)

در حالت سوم، یعنی روش‌های Actor-Critic تلاش میکنیم از نقاط قوت روشهای مبتنی بر ارزش و مبتنی بر سیاست با ترکیب این دو روش استفاده کنیم:

- **Actor**: یک شبکه سیاست که ا $\pi_\theta(a \mid s)$ تعیین میکند.
- **Critic**: یک شبکه ارزش که تابع ارزش V(s)V(s)V(s)یا $Q(s,a)$ را تخمین میزند و به Actor در بهبود پالیسی کمک میکند.

  - **ایده اصلی** :استفاده از Critic برای هدایت سیاست و تثبیت به‌روزرسانی‌هایActor
  - **مزیت اصلی**:

- در کنار انعطاف‌پذیری روشهای مبتنی بر سیاست و توانایی افزودن قیود که قبلتر صحبت کردیم، از واریانس کم روشهای مبتنی بر ارزش هم بهره میبریم. (چرا که عموما روشهای مبتنی بر سیاست دارای واریانس و تغیرات هستند و با استفاده از دو شبکه میتوان این واریانس را کمتر و کنترل کرد)

  - **الگوریتم‌های رایج**:

- **A2C (Advantage Actor-Critic)** : استفاده از تابع مزیت $A(s,a) = Q(s,a) - V(s)$
- **DDPG (Deep Deterministic Policy Gradient)**: برای مجموعه اکشن پیوسته.
- **SAC (Soft Actor-Critic)**: شامل به حداکثر رساندن آنتروپی برای اکتشاف که در کلاس مورد بحث قرار گرفت و دیدیم برای یادگیری اجتماعب میتواند بسیار کارآمد باشد.

  - **معایب**:

- پیاده‌سازی روشهای actor-critic با پیچیدگی بیشتری همراست.
- ممکن است محدودیت های هر دو رویکرد فردی را به ارث ببرد (مانند ناپایداری).

سوال دوم:

الگوریتم Dueling DQN بر اساس DQN معمول ساخته شده است و معماری شبکه عصبی را برای بهبود عملکرد و کارایی یادگیری در وظایف تقویتی اصلاح میکند. تفاوت اصلی این دو در نحوه محاسبه مقادیر Q است که در آن ارزیابی ارزش حالت (state value) و تابع مزیت (advantage function) جدا میشود.

تفاوت روش Dueling DQN و DQN:

درDQN ، الگوریتم برای هر عمل a در یک حالت s، مقدار واحدی $Q(s,a)$ را یاد می‌گیرد. اما درDueling DQN ، این مقدار به دو بخش تقسیم میشود:

ارزش حالت$\boldsymbol{V(s)}$:میزان خوب یا بد بودن حالت فعلی s، صرف نظر از اینکه چه عملی انجام شود.

تابع مزیت (advantage function) : $\boldsymbol{A(s,a)}$ میزان بهتر یا بدتر بودن یک عمل خاص a نسبت به سایر اعمال در همان حالت s.

اگر یک عمل بسیار بهتر از باقی عملها باشد، $\boldsymbol{A(s,a)}$ مقدار مثبت بزرگی خواهد داشت.

اگر بدتر باشد، مقدار منفی خواهد داشت.

اگر همه اعمال به یک اندازه خوب باشند، $\boldsymbol{A(s,a)}$ برای همه اعمال نزدیک به صفر خواهد بود.

سپس مقدار Q به صورت زیر محاسبه میشود:

$$Q(s,a) = V(s) + A(s,a)$$

یا به عبارتی میتوان گفت:

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s).$$

که براساس منبع داده شده به صورت دقیق تر میتوان به صورت زیر نیز مقدار Q را نوشت:

$$Q(s,a) = V(s) + (A(s,a) - \frac{1}{|A|}\sum_{a'} A(s,a'))$$

این فرمول تابع مزیت را نرمالسازی میکند تا بر Q غلبه نکند، درواقع برای اطمینان از اینکه تجزیه مقدار Q منحصر به فرد باشد، تابع مزیت با کم کردن میانگین آن در میان تمام اعمال موجود در حالت نرمالسازی میشود. این کار پایداری محاسباتQ را تضمین میکند.

مزایایDueling DQN :

۱. تعمیم بهتر در بین اعمال :جداسازی $\boldsymbol{V(s)}$ و $\boldsymbol{A(s,a)}$به شبکه اجازه میدهد بر یادگیری حالتهای مهم تمرکز کند، خصوصا زمانی که برخی از اعمال خاص بی اهمیت هستند.

۲. همگرایی سریع‌تر :با یادگیری ارزش حالتها به‌طور مستقل، شبکه از یادگیری اطلاعات تکراری درباره اعمال در حالتهای مشابه جلوگیری میکند.

عملکرد بهتر در محیطهای پیچیده :آزمایشها (مانند بازی های آتاری) نشان داده اند که Dueling DQN نسبت به DQN سنتی عملکرد بهتری به ویژه در وظایفی با فضای عمل بزرگ دارد.

سوال سوم:

در سوال اول، به بررسی و معرفی کلی روشهای **actor-critic** پرداختیم و دیدیم که در ای چارچوب:

- عامل (**Actor**): سیاست (**Policy**) را یاد می‌گیرد تا تصمیم‌گیری کند.
- منتقد (**Critic**): ارزش یا کیفیت اعمال ممکن عامل را ارزیابی میکند.

الگوریتم **A2C (Advantage Actor-Critic)**

الگوریتم A2C یک نسخه پیشرفته از الگوریتم actor-criti است که مانند روش Dueling DQN مفهوم تابع مزیت را معرفی میکند. این تابع تفاوت بین ارزش یک عمل خاص و مقدار میانگین ارزش وضعیت را اندازه‌گیری میکند. استفاده از این اطلاعات به الگوریتم کمک میکند تا تمرکز بیشتری بر روی اقداماتی که ارزش بالاتری دارند داشته باشد.تابع مزیت به صورت زیر تعریف میشود: (همانند روش قبل)

$$\boldsymbol{A(s,a) = Q(s,a) - V(s)}$$

همانطور قبلتر هم گفتیم **تابع مزیت** میزان برتری یک عمل نسبت به مقدار پایه (ارزش حالت) را تعیین میکند. این تابع، واریانس در به‌روزرسانی های گرادیان سیاست را کاهش میدهد و به پایداری و کارایی یادگیری کمک می‌کند. در:A2C

- **بازیگر (سیاست)** از مزیت برای هدایت به‌روزرسانی ها استفاده میکند و اعمالی با بازده نسبی بالاتر را ترجیح میدهد.
- **منتقد (تابع ارزش)** V(s)را تخمین میزند تا مقدار پایه را فراهم کند.

گرادیان سیاست با استفاده از فرمول زیر به‌روزرسانی میشود:

$$Policy\ Gradient = \nabla_{\theta} log\pi(a \mid s;\theta) \cdot A(s,a)$$

تفاوت A2C با Dueling DQN :

۱. **معماری و روش:**
   - **A2C**: ترکیبی از بازیگر و منتقد است. منتقد بازخورد (مزیت) را برای بازیگر فراهم میکند تا سیاست خود را بهبود دهد.
   - **Dueling DQN**: معماری شبکه Q را تغییر داده و آن را به دو بخش تقسیم میکند تا ارزش حالت V(s) و مزیت A(s,a) را جداگانه تخمین میزند و سپس برای محاسبه Q(s,a) ترکیب کند.

۲. **استفاده از تابع مزیت:**
   - **A2C**: مزیت را مستقیما با استفاده از $A(s,a) = Q(s,a) - V(s)$ محاسبه کرده و برای تنظیم به روزرسانی سیاستها استفاده میکند.
   - **Dueling DQN**: مقدار Q(s,a) را به V(s) و A(s,a) تجزیه میکند تا نمایش بهتری از حالت و اعمال خاص ارائه دهد.

۳. **فضای اعمال:**
   - **A2C**: در هر دو فضای اعمال گسسته و پیوسته به خوبی عمل میکند.
   - **Dueling DQN**: عمدتا برای فضای اعمال گسسته طراحی شده است.

۴. **مکانیزم پایداری:**
   - **A2C**: با استفاده از تابع مزیت برای کاهش واریانس و ماهیت روی سیاست خود به پایداری میرسد.
   - **Dueling DQN**: با جدا کردن ارزش حالت و مزیت اعمال خاص، پایداری را به دست میاورد و از خطاهای بیش‌براوردی جلوگیری میکند.

بخش دوم:

سوال چهارم:

محیط Cart Pole v1 یکی از محیطهای موجود در کتابخانه Gymnasium است. در این محیط، یک میله به یک چرخ دنده متصل شده که روی یک ریل بدون اصطکاک حرکت میکند. هدف این است که میله را با اعمال نیرو به چپ یا راست روی چرخ دنده در حالت عمودی نگه داریم.

- **فضای اعمال حرکت (Action Space)**
  - نوع: Discrete(۲)
  - مقادیر:
    - 0: چرخ‌دنده را به سمت چپ هل دهید
    - 1: چرخ‌دنده را به سمت راست هل دهید.

نیروی اعمال شده به چرخ دنده مقدار ثابتی دارد، اما بسته به زاویه میله، میزان سرعت تغییر میکند زیرا مرکز ثقل میله بر انرژی موردنیاز برای حرکت چرخ‌دنده تأثیر میگذارد.

- **فضای مشاهده (Observation Space)**
  - نوع: Box([-4.8, -inf, -0.418, -inf], [4.8, inf, 0.418, inf], (4,), float32)
  - شکل: (۴,)
  - پارامترها:

- موقعیت چرخ‌دنده: محدوده [-۴٫۸, ۴٫۸] متر.
- سرعت چرخ‌دنده: محدوده (-∞, ∞) متر بر ثانیه.
- زاویه میله: محدوده [-۰٫۴۱۸, ۰٫۴۱۸] رادیان (±۲۴ درجه).
- سرعت زاویه‌ای میله: محدوده (-∞, ∞) رادیان بر ثانیه.

- **نکات مربوط به پایان اپیزود:**

- اپیزود خاتمه می‌یابد اگر:
  - موقعیت چرخ‌دنده بیشتر از ±۲٫۴ متر شود.
  - زاویه میله بیشتر از ±۰٫۲۰۹۵ رادیان (~±۱۲ درجه) شود.
  - تعداد گام های اپیزود به حداکثر مقدار (۵۰۰ گام) برسد.

- **پاداش‌ها (Rewards)**

- سیستم پاداش پیش فرض:
  - برای هر گامی که میله در حالت عمودی باقی بماند، مقدار +۱ پاداش داده میشود (شامل گام خاتمه).
  - حد آستانه پاداش پیش فرض ۵۰۰ است.
- **سیستم پاداش جایگزین (sutton_barto_reward=True):**
  - برای هر گامی که اپیزود خاتمه نیافته، مقدار ۰ پاداش داده میشود.
  - برای گام خاتمه مقدار -۱ پاداش داده میشود.
  - حد آستانه پاداش برای این حالت برابر ۰ است.

- **وضعیت شروع (Starting State)**

- تمام مقادیر مشاهده ها به‌طور پیش‌فرض از محدوده‌ی تصادفی (-۰٫۰۵, ۰٫۰۵) نمونه‌برداری میشوند.
- با استفاده از پارامتر options هنگام بازنشانی، میتوان محدوده ی شروع را تغییر داد.

Action Space	`Discrete(2)`
Observation Space	`Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)`
import	`gymnasium.make("CartPole-v1")`

سوال پنجم:

استفاده از الگوریتم های کلاسیک یادگیری تقویتی (مانند Q-Learning یا SARSA) در محیط CartPole-v1 به دلایل زیر چندان مناسب نیست:

۱. فضای پیوسته مشاهدات

- محیط CartPole-v1 دارای فضای مشاهدات پیوسته است (مانند موقعیت ها، سرعت ها، زوایا)، که این موضوع برای الگوریتم‌های کلاسیک که به فضای حالت گسسته متکی هستند، مناسب نیست:
- الگوریتم‌های کلاسیک مانند Q-Learning به یک جدول Q نیاز دارند که حالت‌ها را به اقدامات نگاشت کند. در فضای پیوسته باید حالت ها گسسته شوند، که منجر به از دست رفتن دقت، رشد نمایی فضای حالت (انفجار حالت)، عملکرد ضعیف در تعمیم دهی به حالت‌های دیده نشده میشود.

۲. ابعاد و پیچیدگی محیط

فضای مشاهده محیط شامل ۴ بُعد است:

۱. موقعیت چرخ دنده.

۲. سرعت چرخ دنده.

۳. زاویه میله.

۴. سرعت زاویه ای میله.

گسسته کردن این ابعاد با دقت بالا منجر به ایجاد فضای حالت بسیار بزرگ میشود، که این موضوع الگوریتم های کلاسیک را از لحاظ محاسباتی ناکارآمد و غیرعملی میکند.

۳. عدم تعمیم مناسب

- الگوریتم های کلاسیک در تعمیم دهی به حالت های مشابه ضعیف عمل میکنند:
- در محیطهای دارای فضای حالت پیوسته مانند CartPole، حالتهای مشابه اغلب به اقدامات مشابه نیاز دارند. الگوریتم های کلاسیک نمیتوانند این تعمیم را به‌صورت ذاتی انجام دهند و هر حالت گسسته را به‌طور مستقل در نظر میگیرند.
- اما از طرف دیگر روش های مدرن مانند شبکه های عصبی در DQN یا الگوریتم های گرادیان سیاست (مانند PPO یا A2C) با استفاده از تقریب‌گرها (مانند شبکه های عصبی) به خوبی حالتهای پیوسته تعمیم میدهند.

۴. بی ثباتی در وظایف کنترلی

محیط CartPole یک وظیفه کنترلی است که در آن خطاهای کوچک در انتخاب اقدام میتواند سیستم را ناپایدار کرده و منجر به شکست شود. الگوریتم های کلاسیک در موارد زیر با مشکل مواجه میشوند:

- پیشبینی دقیق اقدام بهینه در حوزه های پیوسته.
- سازگاری با ماهیت پویا و حساس وظایف کنترلی که نیاز به دقت بالایی دارند.

۵. پیچیدگی نمونه برداری

الگوریتم های کلاسیک اغلب برای همگرا شدن نیاز به تعاملات زیادی با محیط دارند:

- در محیط های پیوسته، گسسته کردن حالات، باعث ایجاد نویز میشود و پیچیدگی نمونه برداری را افزایش میدهد.
- الگوریتم های مدرن مانند DQN یا PPO با استفاده از حافظه بازپخش (Replay Buffer)، بهبودهای دسته ای، و گرادیان نزولی تصادفی یادگیری را کارآمدتر میکنند.

۶. عدم مقیاس پذیری

الگوریتم های کلاسیک به خوبی برای نسخه های پیچیده تر محیط مانند Acrobot، MountainCar یا CartPole InvertedPendulum مقیاس پذیر نیستند:

چرا الگوریتم‌های مدرن بهتر هستند؟

شبکه های عصبی: الگوریتم های مدرنی مانند DQN و PPO از شبکه های عصبی برای تقریب تابع ارزش یا سیاست استفاده میکنند و فضای حالت پیوسته را به خوبی مدیریت میکنند.

بهینه سازی سیاست: الگوریتم هایی مانند PPO به‌طور مستقیم سیاست را بهینه میکنند و از مشکلات گسسته کردن مقادیر Q جلوگیری میکنند.

حافظه بازپخش: DQN و روش های مشابه از حافظه بازپخش برای یادگیری کارآمد از تجربیات گذشته استفاده میکنند.

مقیاس‌پذیری: این روش‌ها به خوبی به مسائل پیچیده تر و محیط های بزرگتر قابل گسترش هستند.

بخش سوم:

ایده کلی از لینک است اما پس از مطالعه لینک مورد نظر، پیاده سازی انجام شده است.

سوال ششم:

الف) در فایل نوت بوک

ب)

```
BATCH_SIZE = 128
GAMMA = 0.99
EPS_START = 0.9
EPS_END = 0.05
EPS_DECAY = 1000
TAU = 0.005
LR = 1e-4
```

ج) در پوشه مربوطه قرار گرفته اند.

سوال هفتم:

الف) در فایل نوت بوک

ب)

```
BATCH_SIZE = 128
GAMMA = 0.99
EPS_START = 0.9
EPS_END = 0.05
EPS_DECAY = 1000
TAU = 0.005
LR = 1e-4
```

ج) در پوشه مربوطه قرار گرفته اند.

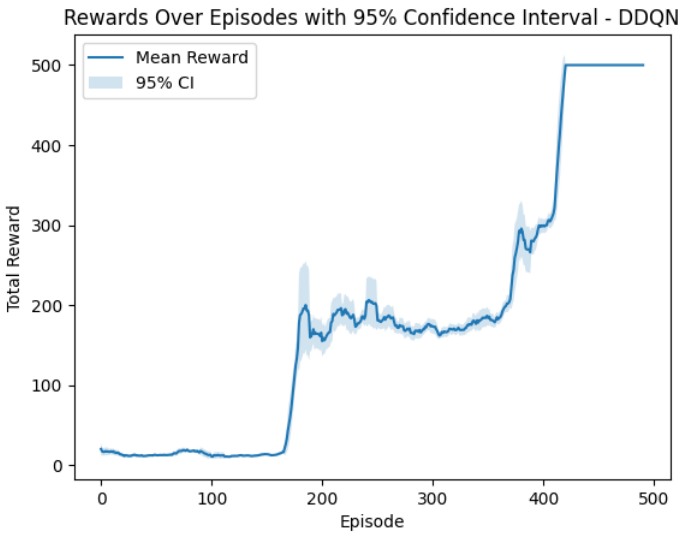
سوال هشتم:

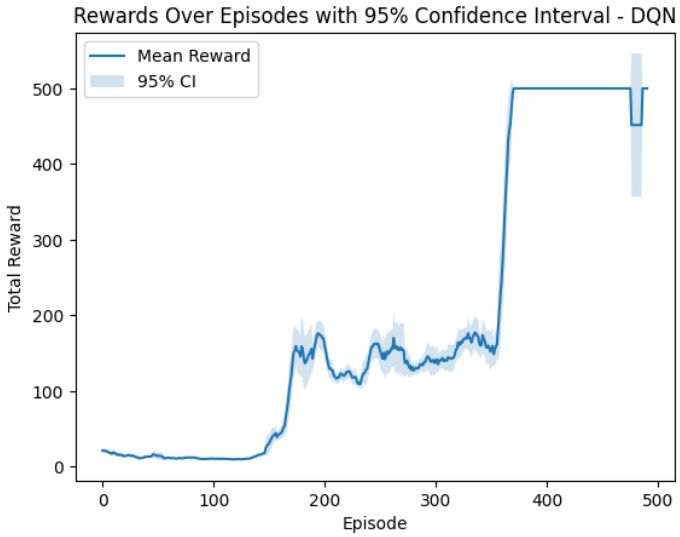
با توجه به نمودار پاداش زیر میتوان دید تغیرات DDQN به نسبت نوسانات کمتری دارد و با شیب ثابت تری نسبت به DQN رشد میکند و در اواسط یادگیری در بازه بین ۱۵۰ تا ۳۵۰ به صورت کلی بهتر عمل کرده است.

الگوریتم DQN نوسانات بیشتری دارد و و رشد در مجموع کند تری نسبت به DDQN دارد.

نتایج به طور منطقی با تئوری الگوریتمها مطابقت دارند:

- DDQN به دلیل کاهش خطای بیش براورد در مقایسه با DQN، به یادگیری سریع تر و پایدارتر دست میابد.
- DQN با وجود نوسانات بیشتر، در نهایت به همان پاداش بهینه دست میابد، اما به طور کلی در طول یادگیری کندتر و پر نوسانتر است.





بخش چهارم:

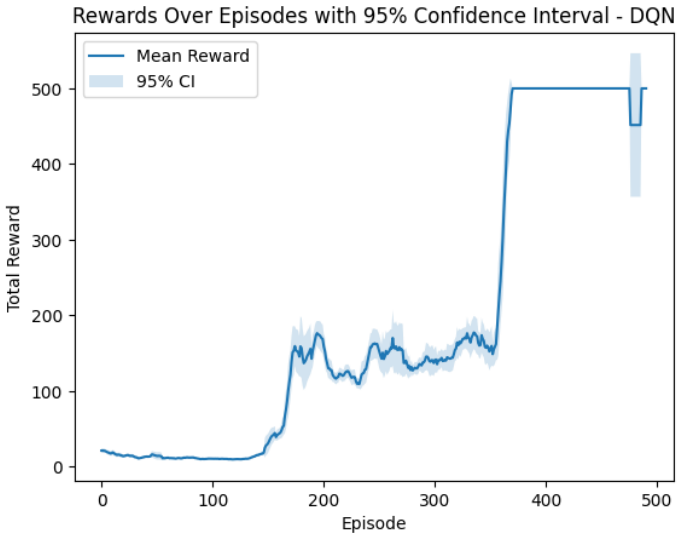
سوال نهم:

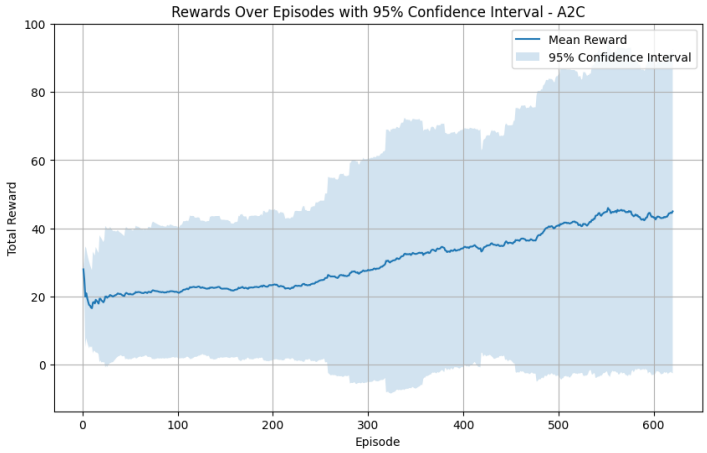
الف) در فایل نوت بوک

ب)

```
    "learning_rate": 1e-4,
    "n_steps": 5,
    "gamma": 0.99,
    "gae_lambda": 1.0,
    "ent_coef": 0.0,
    "vf_coef": 0.5,
    "max_grad_norm": 0.5,
    "learning_rate": 1e-4,
```

سوال دهم:





همانطور که انتظار میرفت میبینیم که **DQN** برای محیط **CartPole-v1** عملکرد بهتری دارد، زیرا با فضای عمل گسسته سازگارتر است و به سرعت سیاست بهینه را یاد میگیرد و از طرف دیگر **A2C** به دلیل ناپایداری ذاتی اش در مسائل ساده و احتمالا تنظیمات غیر بهینه ی هایپرپارامترها یا زمان ناکافی برای آموزش عملکرد ضعیف تری دارد.

	DQN	A2C
رشد پاداش	**رشد سریع پس از حدود ۲۰۰ اپیزود**	**افزایش بسیار اهسته و تدریجی**
عملکرد نهایی	**دستیابی به حداکثر پاداش (۵۰۰)**	**متوقف شدن در پاداش‌های پایین (~۵۰)**

تمامی نتایج و تحلیل ها برای تعداد محدود ران (به دلیل زمان بر بودن زمان اجرا) و برای هایپر پارامترهای گفته شده، میباشند و ممکن است در مواردی کمی تفاوا جزئی دیده شود.

منابع:

بخش اول:

- toDeep_Reinforcement_Learning_Hands-On_-_A_practical_and_easy-to-follow_guide_to_RL_from_Q-learning_and_DQNs_to_PPO_and_RLHF
- Dueling Network Architectures for Deep Reinforcement Learning
- Asynchronous Methods for Deep Reinforcement Learning
- Actor-Critic Algorithm in Reinforcement Learning
- Gymnasium Documentation
- https://stable-baselines3.readthedocs.io/en/master/guide/quickstart.html
- https://github.com/iocfinc/A2C-CartPole/blob/master/A2C-CartPole.ipynb